RFC: Host the OHSU Simulation Database on Managed AWS Services

Field	Value
Author	Kenneth Robertson
Date	2026-10-20
Status	Draft - AWS approval pending
Decision Date	To be completed when decided
Deciders	OSU Capstone team and OHSU project partner
Superseded By	None

Problem and Context

The team needs to decide where and how to host the OHSU Simulation Database. Our architecture worksheet classifies web hosting as hard to reverse because moving an application and its data requires redeployment, new configuration, migration, security review, and possible downtime. The current direction is AWS, pending OHSU approval. Related team decisions already point toward a React and TypeScript frontend, a Node.js and Express backend, PostgreSQL, OIDC-based OHSU single sign-on, and Amazon S3 for uploaded files.

Hosting determines account ownership, secret storage, PostgreSQL and S3 connectivity, logging, backups, OIDC callback URLs, and transfer to OHSU. The decision is needed before environment-specific assumptions spread through the codebase.

If the team does nothing, development can continue locally, but there will be no stable environment for partner testing, authentication, or end-to-end uploads. Delaying compresses time for backup, rollback, permissions, and handoff testing. No production-like resources should be created until OHSU confirms the provider, owner, region, data classification, and budget.

Options

Option A: Managed AWS Deployment

Package the Node.js/Express application as a container and deploy it through Amazon ECS Express Mode. Use Amazon RDS for PostgreSQL and Amazon S3 for uploaded documents. ECS Express Mode provisions an ECS service on Fargate together with load balancing, networking, health checks, and auto-scaling resources [2]. RDS supplies managed database backups and point-in-time recovery within the configured retention period [3].

For the first release, Express could serve the compiled React application and REST API from one container. The database should not be public, and secrets must come from approved configuration rather than committed files.

This aligns with the intended use of S3 and reduces operating-system administration. Costs include containerization, IAM and networking knowledge, recurring charges, and dependence on OHSU providing an AWS environment.

Option B: Self-Managed AWS EC2 Deployment

Run the application on an EC2 virtual machine, with PostgreSQL locally or in RDS, while using S3 for files. EC2 gives direct control over the operating system, proxy, runtime, processes, and network configuration.

AWS states that EC2 customers manage the guest operating system, patches, installed software, and security groups [4]. The team would also configure process supervision, TLS, monitoring, rollback, and recovery. Hosting PostgreSQL on the same instance couples application and database failure. This flexibility creates more work before and after handoff.

Option C: Do Nothing - Remain Local Until Hosting Is Assigned

Do not choose or configure a cloud deployment during the current phase. Keep the application runnable through documented local containers and wait for OHSU to supply an approved hosting environment.

This avoids unauthorized spending and the wrong provider commitment. Its cost is no shared staging environment and delayed discovery of HTTPS, OIDC, permissions, networking, upload, logging, and deployment problems.

Analysis

Complexity and Maintainability

Option A moves more work to managed services. ECS Express Mode creates Fargate compute, load balancing, security groups, networking, and scaling policies [2]. The team still owns the container, application security, IAM, monitoring, and cost control. Option B adds operating-system maintenance. Option C postpones integration work.

AWS has closed App Runner to new customers and recommends ECS Express Mode instead [5]. Therefore, App Runner should not be selected simply because older tutorials describe it as the easiest managed option.

Security and Confidentiality

Managed hosting does not transfer authorization or data-classification responsibility to AWS. The team must enforce OIDC and roles, restrict RDS and S3 through least privilege, encrypt traffic, and avoid logging confidential content. AWS protects the infrastructure while customers remain responsible for configuration, data, identities, and application controls [4].

Option A reduces the surface the team must patch directly, but a mistaken IAM policy can still expose data. Option B adds operating-system and server-hardening responsibilities. Under every option, only synthetic data may be used until OHSU approves the environment for any partner-controlled material.

Performance, Reliability, and Cost

The repository has no measured workload, expected user count, uptime target, or budget, so this draft claims no cost or speed advantage. ECS Express Mode can scale tasks, while RDS supports automated backups [2][3]. EC2 requires the team to build more operational controls.

Managed load balancing and a continuously available database create recurring costs. The team should estimate costs only after OHSU confirms region, availability, storage, retention, and expected traffic.

Reversibility

Containerization and repository-managed migrations improve portability. S3 access should sit behind a storage interface, and infrastructure should be defined as code. Database exports, file manifests, and documented environment variables are required for migration.

Recommendation and Verification Plan

I recommend Option A, conditional on written OHSU approval. The initial topology should be one containerized Node.js/Express service in ECS Express Mode, an RDS PostgreSQL database reachable only through approved network paths, and a private S3 bucket for uploaded files. This matches the team's current stack while limiting server administration.

The team will know the decision works when:

- A clean deployment can be created from repository instructions without manual changes to application code.
- HTTPS health checks pass and a previous container image can be redeployed as a rollback.
- The application connects to RDS without making the database publicly accessible.
- A synthetic file can be uploaded, authorized, downloaded, versioned or replaced as designed, and removed without exposing the bucket publicly.
- OIDC login succeeds with an approved callback URL, and unauthorized users cannot access protected routes or objects.
- An RDS backup is restored into a separate test database and record counts plus application checks match the source.
- Logs contain request identifiers and operational errors but not secrets, tokens, or full simulation documents.
- A synthetic end-to-end test meets response-time and concurrency targets that the team and partner set before testing.
- A teammate can follow the handoff guide to locate resources, rotate credentials, deploy, roll back, and disable the environment.

The first deployment should use synthetic data in a nonproduction environment. Production-like data must wait for partner security approval. If AWS is rejected, the team should reopen this RFC and compare approved providers using the same acceptance criteria rather than silently substituting a host.

Constraints

OHSU approval, account ownership, billing, region, identity integration, retention, backup, and incident-response requirements are unresolved constraints. The public repository must not contain credentials, internal endpoints, partner-confidential documents, production exports, or identifiable patient, learner, faculty, or staff information [1]. Third-party packages and container images must have compatible licenses. The team must clarify whether OHSU, OSU, or the student team owns the cloud resources and domain before deployment.

Delegation and Validation

I used ChatGPT/Codex to inspect the repository and team worksheet, research current AWS services, compare hosting options, and draft the RFC. The tool did not create AWS resources, estimate an unapproved budget, process partner data, or claim that OHSU has approved AWS. Current AWS documentation was checked because service availability changed: App Runner is closed to new customers, and AWS recommends ECS Express Mode [5].

The repository currently requires teammate review but has no deployed staging environment, infrastructure tests, or proven rollback path. Before accepting the RFC, I will personally confirm the worksheet's architecture choices with the team, open every cited source, and review the proposed topology with the OHSU partner. I will stop before account creation, data import, or SSO registration until

the responsible organization approves those actions.

Decision

Pending. Record acceptance, rejection, or supersession only after OHSU confirms whether AWS is permitted, along with the decision date and chair.

References

1. OSU Capstone OHSU Database, “OHSU Simulation Database Agent Guide,” 2026. https://github.com/robekenn/OSU-Capstone-OHSU-Database/blob/main/AGENTS.md
2. Amazon Web Services, “Amazon ECS Express Mode,” 2026. https://docs.aws.amazon.com/AmazonECS/latest/developerguide/express-service-overview.html
3. Amazon Web Services, “Introduction to Backups - Amazon RDS,” 2026. https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_WorkingWithAutomatedBackups.html
4. Amazon Web Services, “Shared Responsibility - Security Pillar,” 2026. https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/shared-responsibility.html
5. Amazon Web Services, “AWS App Runner Availability Change,” 2026. https://docs.aws.amazon.com/apprunner/latest/dg/apprunner-availability-change.html